

Práctica REST MySQL e IoT

Natalia Hernández Piedrahita

Miguel Ángel Diuza

Juan Ospina Tenorio

Ejecución: 24 de septiembre de 2026 · America/Bogota.

Entorno: Ubuntu 22.04 ARM64, Vagrant y VirtualBox.

35 / 35

42 / 42

60 / 60

Pruebas Python aprobadas

Respuestas HTTP correctas

Aserciones Postman/Newman

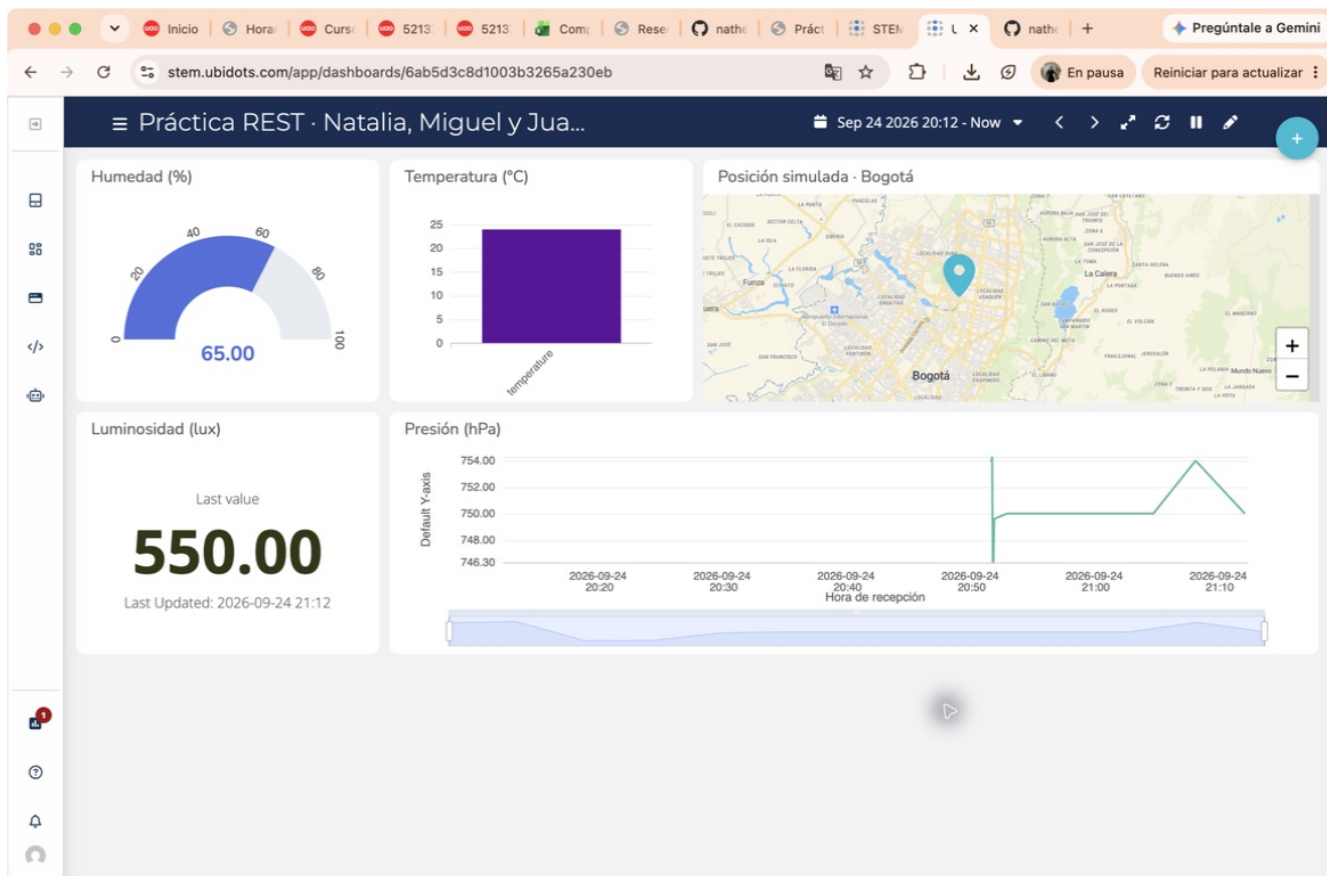


Figura 1. Dashboard real de Ubidots STEM después de la prueba de alerta: cinco variables recibidas y visualizadas.

Estado: APIs de memoria y MySQL, desafío JavaScript y persistencia comprobados. Ubidots recibió las cinco variables, el dashboard se actualizó y el evento envió la alerta. La captura aportada por Natalia confirma la recepción del correo. Los requisitos técnicos están completos.

Repositorio público: <https://github.com/nathernandez1189/practica-rest-mysql-ubidots>

API REST y ejecución sobre Ubuntu

La práctica implementa el recurso **books** con los campos id, title, description y author. Las dos APIs Flask comparten el contrato HTTP; la implementación JavaScript utiliza el mismo esquema MySQL.

Servicio	Almacenamiento	Puerto
Python / apirest.py	Memoria del proceso	5000
Python / apirest_mysql.py	MySQL 8.0.46	5001
JavaScript / server.js	MySQL 8.0.46	5002

Operación	Ruta	Resultado esperado
Consultar	GET /books y /books/<id>	200; 404 si no existe
Crear	POST /books	201 y cabecera Location
Actualizar	PUT /books/<id>	200; validación 400
Eliminar	DELETE /books/<id>	200; 404 si no existe

Se rechazan títulos vacíos, tipos incorrectos y campos desconocidos. Las consultas SQL se parametrizan; se comprobó que una cadena con sintaxis SQL se conserva como texto. Los errores de conexión se devuelven como 503 sin exponer datos internos.

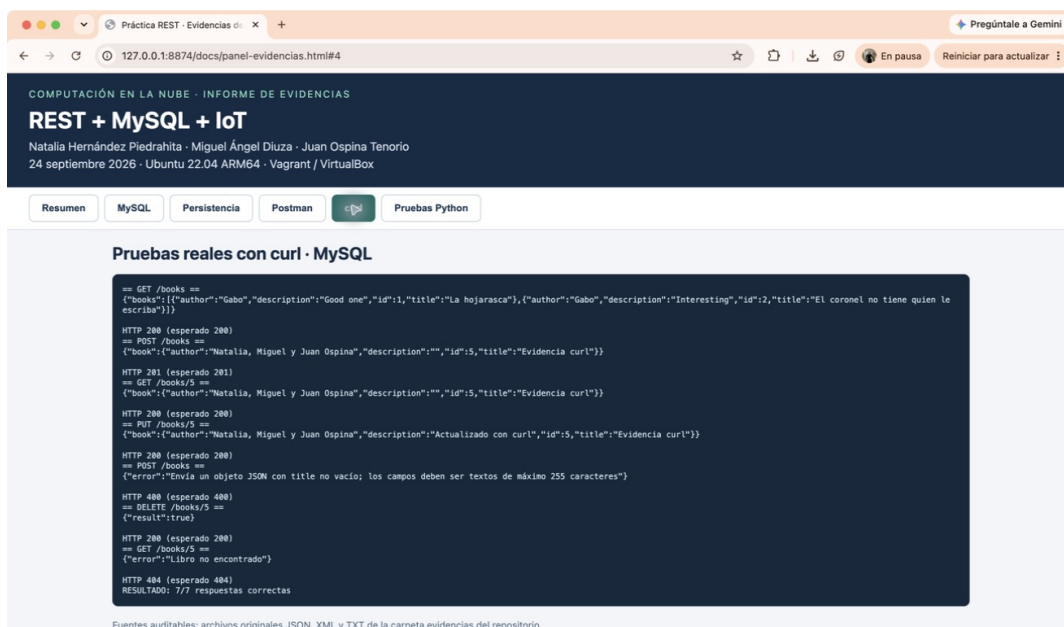


Figura 2. Registro real de curl contra MySQL: siete respuestas esperadas. También se ejecutó el mismo flujo en memoria y JavaScript.

Las APIs se instalaron en /home/vagrant/practica-rest-equip, sobre servidorUbuntu (192.168.100.3). La alternativa servidorRest del Vagrantfile fue validada como configuración; no se creó esa nueva VM.

MySQL y persistencia comprobada

La base exclusiva **rest_equipo** usa una tabla **books** con motor InnoDB y codificación utf8mb4. id es clave primaria autoincremental; title, description y author son VARCHAR(255). La cuenta de la API tiene permisos de lectura y escritura sobre esta base.

Origen	ID de prueba	Tras encender	Interpretación
memory	6	404	Se perdió en memoria
mysql	9	200	Se conservó íntegro
javascript	10	200	Se conservó íntegro

Figura 3. Resultados contrastados antes y después de un apagado real de Ubuntu. El archivo vm-apagada.txt registra el estado poweroff.

Los registros se crearon antes del apagado y se consultaron después del encendido. Se compararon sus campos completos. MySQL mantuvo los libros creados por Python y JavaScript; el registro de memoria devolvió 404.

Antes: 2026-09-25T01:31:28.872574+00:00
Después: 2026-09-25T01:34:11.810735+00:00

Respuesta al ejercicio: apagar la VM conserva los datos confirmados de MySQL en su disco. La memoria del proceso se reinicia. Destruir la VM o borrar su disco es diferente de apagarla y puede eliminar los datos.

Evidencias: persistencia.json, vm-apagada.txt, vm-encendido.txt, mysql-despues-encendido.txt y respaldo-rest_equipo.sql. La copia SQL contiene únicamente libros de esta práctica.

Pruebas reproducibles y Postman

Las comprobaciones se separan por herramienta. Algunas validan el mismo comportamiento desde niveles distintos; no se suman como si fueran requisitos únicos.

Grupo	Resultado	Archivo auditable
Python / pytest	35 aprobadas	evidencias/pytest.txt y pytest.xml
HTTP real	14 por API; 42 en total	evidencias/http-real.json
Postman / Newman	20 por API; 60 aserciones	evidencias/newman-5000/5001/5002.*
curl	7 por API; 21 respuestas	evidencias/curl-*.txt

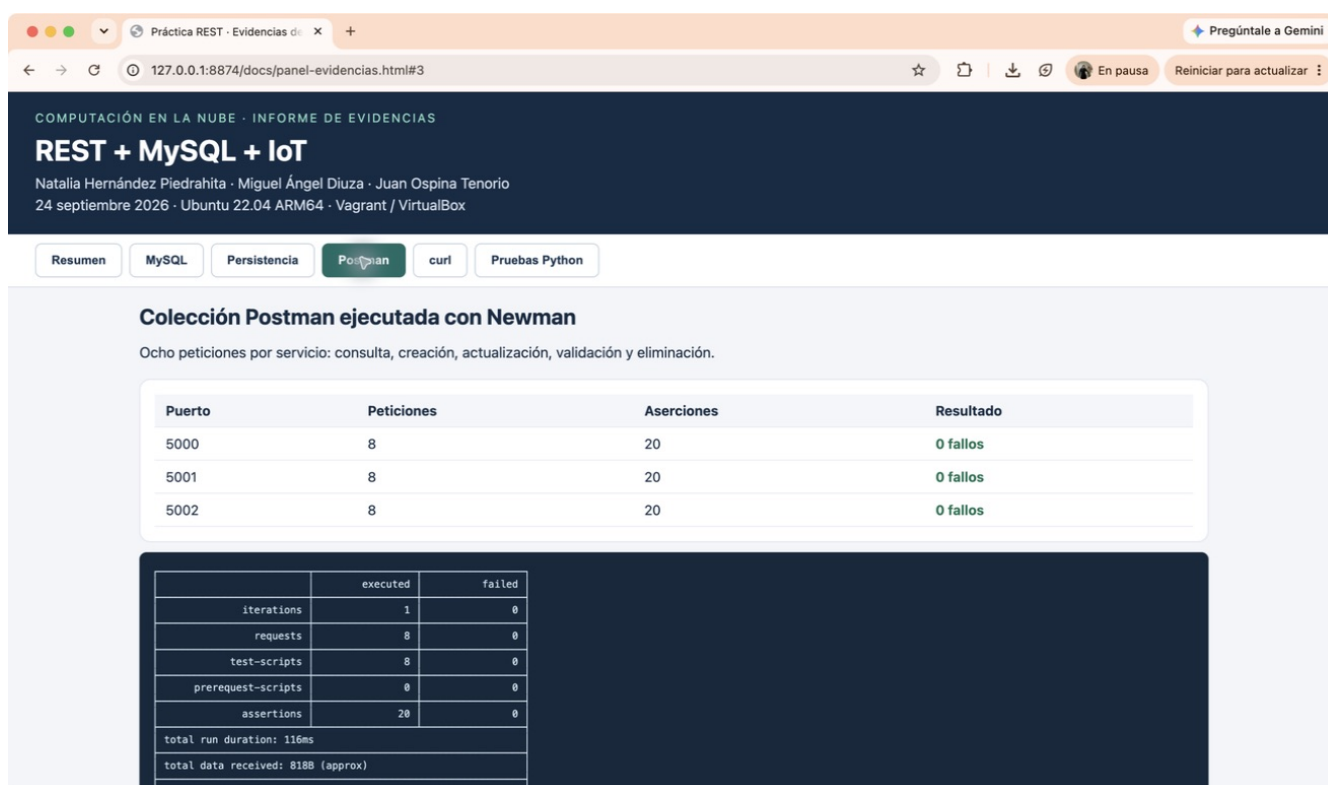


Figura 4. Captura del resumen de Newman: ocho peticiones y veinte aserciones por implementación, sin fallos.

Alcance de Postman: la colección se ejecutó con Newman, el ejecutor oficial de Postman. No se afirma haber utilizado la interfaz gráfica. La colección está lista para importarla y ejecutar Collection Runner.

Cobertura: CRUD completo, lectura de un recurso eliminado, ids inexistentes, títulos inválidos, campos inesperados, caracteres Unicode y texto con sintaxis SQL. Las pruebas Python agregan vaciado total de memoria, ids no reutilizados y respuestas 503 controladas.

La automatización de GitHub repite pruebas Python y pruebas de integración con un servicio MySQL real. El estado vigente debe consultarse en la pestaña Actions del repositorio; los archivos locales conservan el resultado de esta ejecución.

Ubidots: recepción real comprobada

Se configuró una cuenta **Ubidots STEM** y el dispositivo **equipo-rest**. Python envió cinco muestras de cinco variables; cada envío obtuvo HTTP 200 y códigos internos 201. Después se repitió el envío con curl y con la colección Postman ejecutada por Newman.

Variable	Contenido simulado	Widget creado
temperature	Temperatura, 18 a 35 °C	Barras
humidity	Humedad, 35 a 85 %	Gauge / escala 0-100
position	Latitud y longitud ficticias	Mapa de Bogotá
luminosity	100 a 1000 lux; adicional	Indicador numérico
pressure	745 a 755 hPa; adicional	Gráfica de línea

Prueba real	Resultado	Evidencia en evidencias/ubidots/
Python	25 valores recibidos	python-envio-real.txt
curl	5 variables aceptadas	curl-envio-real.json
Postman / Newman	12 aserciones sin fallos	postman-newman.txt y .json
Consulta independiente	5 valores contrastados	lectura-valores-reales.json

El cliente usa requests y HTTPS con validación TLS activa. El token se lee de UBIDOTS_TOKEN y se excluye del repositorio. Se comprueban los códigos internos por variable y los datos guardados mediante una consulta GET. Los dobles de las pruebas unitarias y el modo --dry-run se mantienen separados de estas pruebas reales.

La simulación de posición corresponde a Bogotá; no representa la ubicación de los integrantes. Las muestras son finitas y no quedó un proceso enviando continuamente. El dispositivo, el dashboard y el evento permanecen en la cuenta del equipo.

Reproducir: docs/UBIDOTS.md; scripts/verify_ubidots.py permite ejecutar normal, alerta y recuperacion con lectura de comprobación. Las capturas públicas permiten al profesor revisar el resultado; el dashboard vivo requiere iniciar sesión.

Fuentes oficiales: [consulta de últimos valores](#), [configuración de eventos](#) y [registros de ejecución](#). Consultadas el 24 de septiembre de 2026.

Dashboard y actualización automática

Cinco widgets reales: humedad, temperatura, posición, luminosidad y presión. Las dos capturas muestran la muestra de alerta y la posterior recuperación.

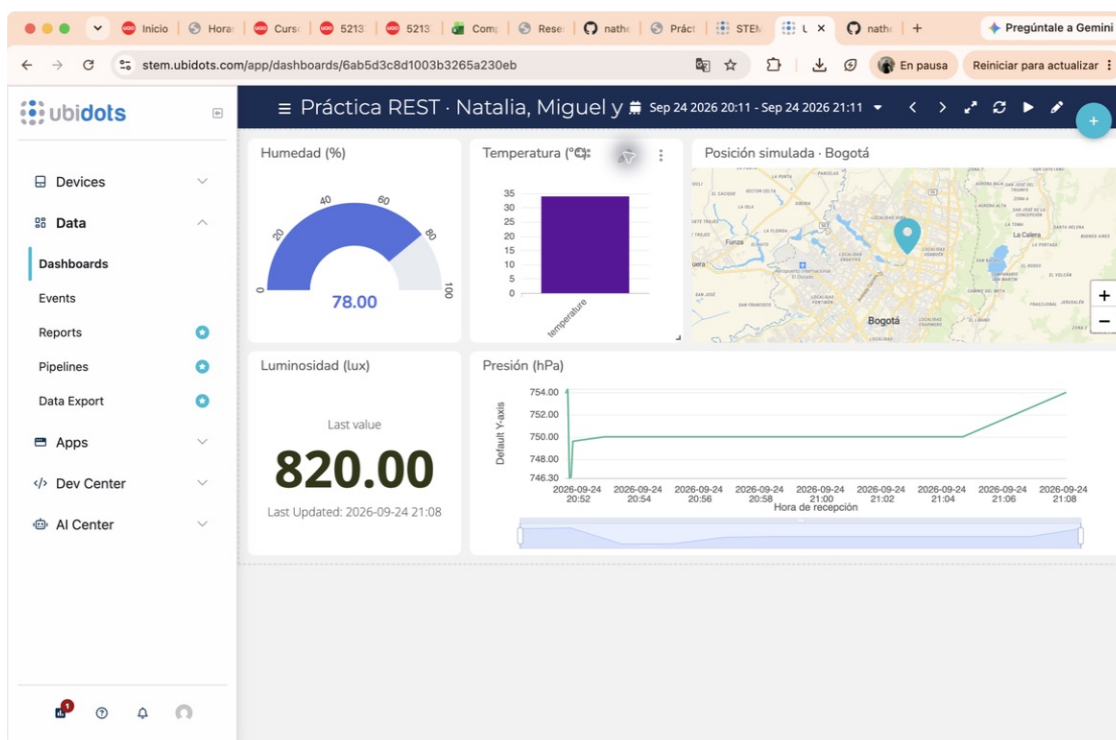


Figura 5. Muestra de alerta: 34 °C, humedad 78 %, luminosidad 820 lux y presión 754 hPa. Mapa con coordenadas ficticias.

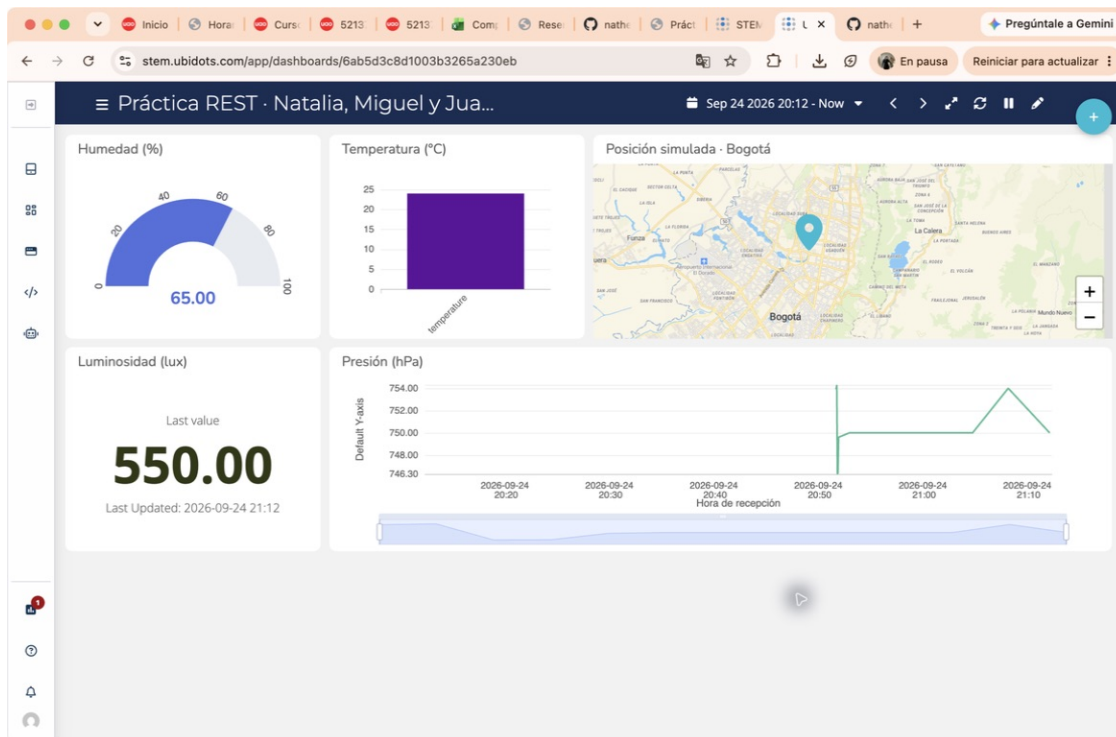


Figura 6. Actualización automática tras enviar la recuperación: 24 °C, 65 %, 550 lux y 750 hPa; marca de tiempo 21:12. La barra lateral se contrajo para ampliar los widgets.

Evento y correo: envío verificado

Se creó **Practica REST - temperatura > 30 C**, con condición temperature > 30, demora cero y horario diario en America/Bogota. La acción envía al correo institucional autorizado por Natalia, sin repetición periódica.

Paso	Resultado
Base: 24 °C	Condición falsa; lectura real verificada
Alerta: 34 °C	Condición verdadera; envío registrado a las 21:08:03
Recuperación: 24 °C	Condición falsa; tablero actualizado a las 21:12

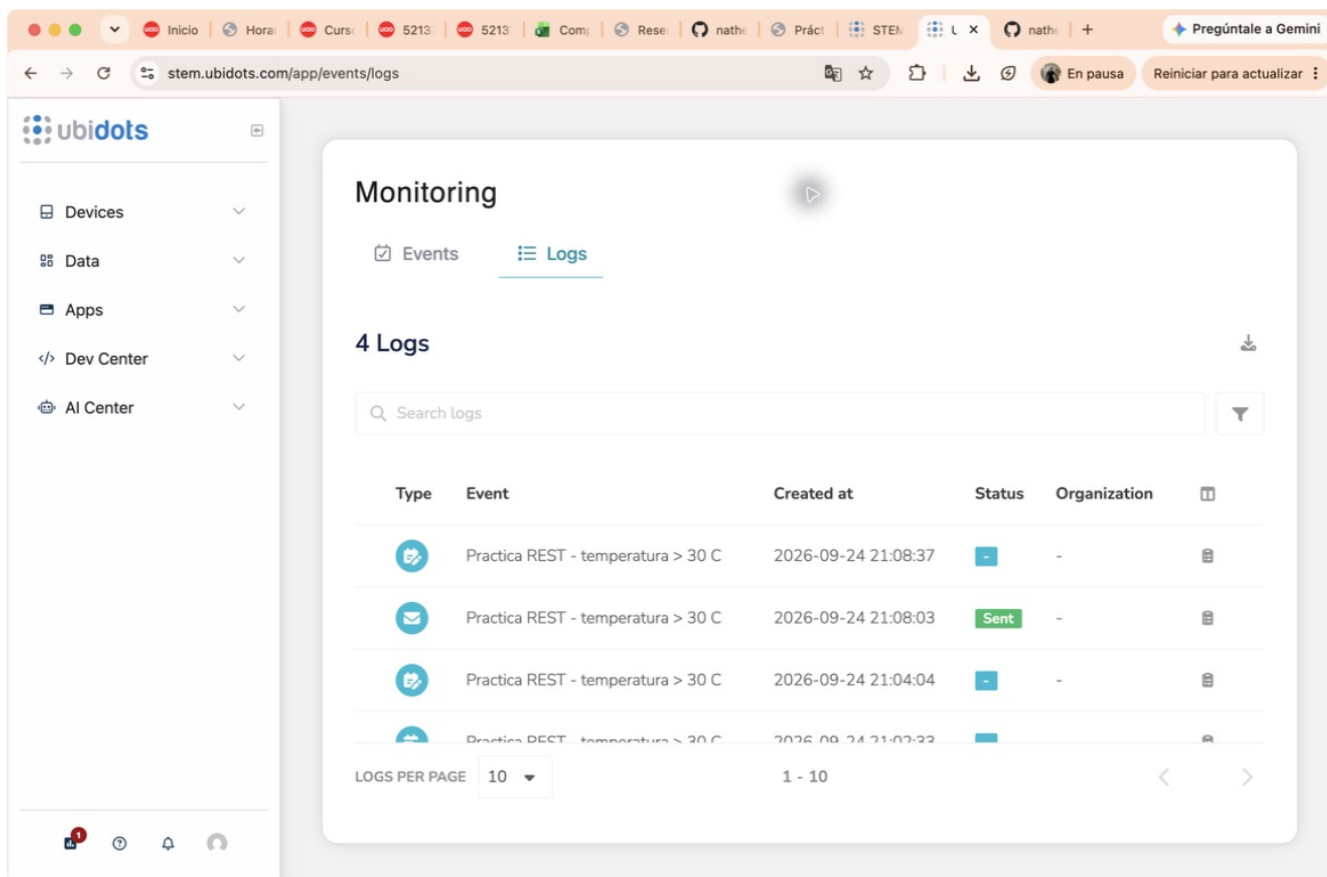


Figura 7. Registro original de Ubidots con estado Sent. Se ocultó la columna Message usando los controles de la aplicación para no publicar la dirección de correo.

Evidencia del envío: evento-logs.json contiene action_emails y successful_execution: true. evento-configuracion.json conserva el umbral y la acción. El correo y el token se ocultan en los archivos publicados.

Recepción confirmada: la captura original aportada por Natalia muestra la alerta en Recibidos a las 9:08 p. m. La hora y el contenido coinciden con el evento registrado. La evidencia se presenta en la página siguiente.

El evento queda activo con temperatura normal de 24 °C. El asunto vigente es Practica REST - Alerta de temperatura del equipo. Se corrigió la ortografía después de la primera prueba; el registro histórico conserva el texto original.

Correo recibido: evidencia de cierre

Natalia aportó esta captura de la alerta recibida. Se observan el remitente **Notifications Ubidots**, la etiqueta **Recibidos**, la hora **9:08 p. m.** y el contenido referido al dispositivo equipo-rest y al umbral de 30 °C.

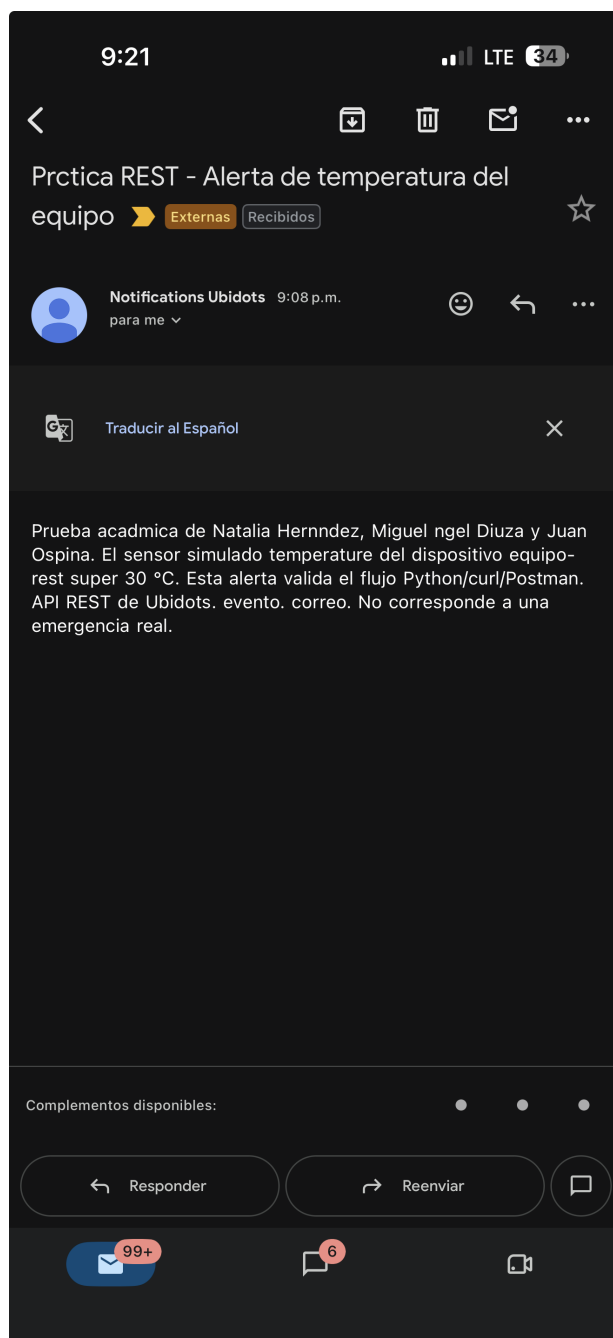


Figura 8. Captura original aportada por Natalia el 24 de septiembre de 2026. La pantalla marca 9:21 y el mensaje 9:08 p. m.; esta última hora coincide con el registro de envío a las 21:08:03.

Se verificó la recepción en la imagen compartida; no se accedió directamente al buzón. El archivo se conserva sin editar y la dirección del destinatario no está desplegada. Su procedencia y huella SHA256 están en evidencias/ubidots/correo-recepcion.json.

El correo conserva el texto de la primera prueba, anterior a la corrección ortográfica ya documentada. Esta evidencia completa el flujo: muestra simulada, recepción en Ubidots, dashboard, evento y correo recibido.

Desafío, reproducción y sustentación

El desafío opcional se implementó en **JavaScript**, usando el módulo HTTP de Node.js y mysql2. Comparte el contrato de libros y la base con Python; se comprobó con las mismas pruebas HTTP, curl, Postman y persistencia.

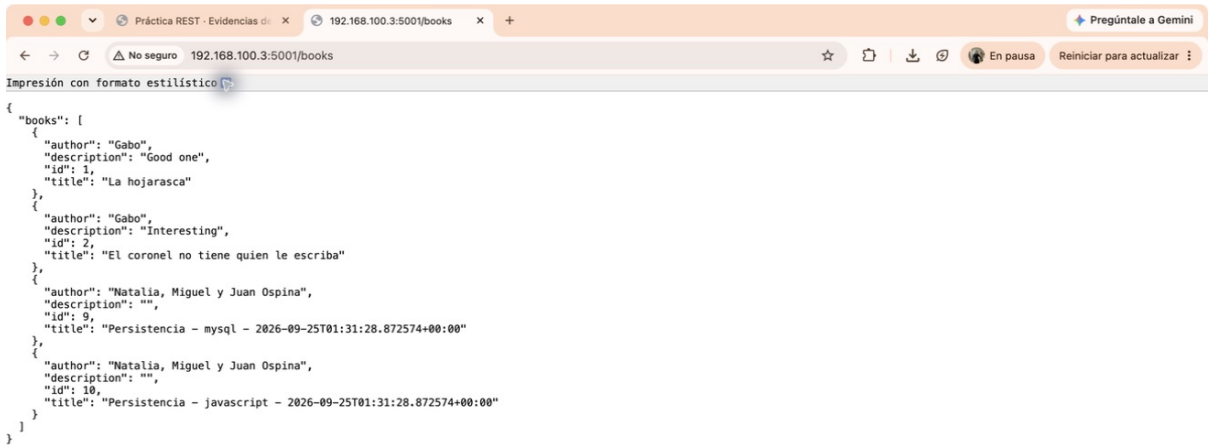


Figura 9. Consulta directa a la API activa en el navegador: libros iniciales y los dos marcadores conservados después del apagado.

Reproducir y revisar

1. Consultar README.md y docs/REPRODUCIR.md.
2. Encender servidorUbuntu desde ~/prueba con Vagrant; entrar por SSH.
3. En /home/vagrant/practica-rest-equipo ejecutar pytest y scripts/verify_http.py.
4. Importar las colecciones de postman/ o repetir Newman.
5. Revisar los registros de evidencias/ y reproducir Ubidots con un token local propio.

Integrante	Propuesta de explicación
Natalia Hernández Piedrahita	API REST, métodos HTTP y pruebas
Miguel Ángel Diuza	MySQL, consultas y persistencia
Juan Ospina Tenorio	Simulador IoT y desafío JavaScript

Este reparto es una propuesta para la sustentación, no una atribución de autoría del código. La guía incluye preguntas probables y respuestas técnicas.

Referencias docentes: guías REST (2024) y REST + MySQL (2023); repositorios de omondragon/APIRestFlask, APIRestFlaskMySQLUbuntu y UbidotsClient. Revisiones y adaptaciones documentadas en docs/REFERENCIAS.md. Los PDFs originales no se redistribuyen.

Entrega: el repositorio queda disponible para el profesor. Su publicación no equivale al envío en CampusDigital; no se ha realizado una entrega en ese portal desde esta tarea.